

Redux

State Management

for JavaScript Applications



What We'll Cover Today

01

What is Redux?

Library, state, pattern

02

Why Redux?

Problems it solves — prop drilling, context issues

03

The 4 Core Terms

Store · Reducer · Action · Dispatch

04

The Redux Flow

How all 4 terms work together

05

Practical Code

Plain JS — build a user account from scratch



01

What is Redux?

Understanding the library and the problem it solves

What is Redux?

A JavaScript library for state management

Redux is a JavaScript library that makes **State Management** easy and predictable.

JavaScript Library

Pre-written JS code you install and import — like jQuery or Axios

A Pattern

Fixed rules for how data changes — every project looks the same

Framework-agnostic

Works with React, Angular, Vue, or plain JavaScript

What is State?

The data your application uses and needs to manage

State = Data

Any data your app uses is called state

Numbers

Strings

Arrays

Objects

Booleans

Management =

Controlling that data

- Fetch it when needed
- Modify it correctly
- Delete it safely
- Sync it everywhere it's used



02

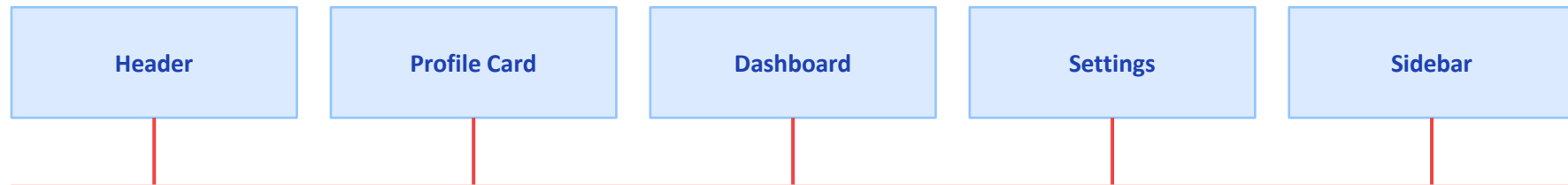
Why Redux?

The real problems it was built to solve

The Synchronization Problem

Why managing state is harder than it looks

Imagine: A user's name shown in 5 places. User updates the name. All 5 must update instantly.



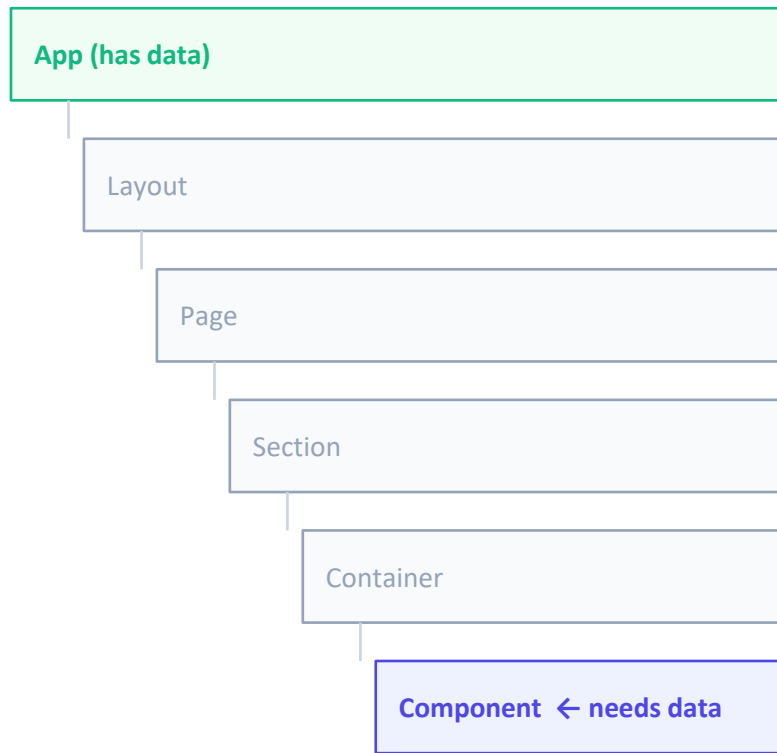
✗ Without a system:

You'd have to manually update each component. Miss one — data is out of sync. Bug introduced.

✓ With Redux:

Problem 1 — Prop Drilling

Passing data through layers that don't need it



Props passed through every level ↑

Prop Drilling

- Components that don't need data still receive it
- Refactoring one prop means changing every level
- Hard to debug — where did this prop come from?
- Becomes unmanageable with 50+ components

Problem 2 — Context API Isn't Enough

Built-in solution, but with serious trade-offs

Context API

Seems like the solution...

- Built into React — no install
- Avoids prop drilling
- Global data access

BUT →

The Reality

✗ Re-render storm

Every context change re-renders ALL consumers — even unrelated ones

✗ No pattern

No standard structure — every developer solves it differently

✗ Hard to debug

No built-in tools to track what changed and when

Without Redux — 5 Real Problems

Why we need a proper state management solution

Prop Drilling



Data passed through 10+ levels to reach one child

Context Re-render Storm



ALL consumers re-render on any context change

No Enforced Pattern



Every developer manages state differently — messy code

No Single Source of Truth



Data scattered across many components — hard to debug

Unpredictable State Changes



Any component can modify shared state — bugs hide

Redux Solves This With:



One Central Store

All data in one place — no drilling



Strict Update Rules

State only changes via dispatch → reducer



Same Pattern Always

Every Redux project follows identical structure



Predictable Updates

Every change is traceable and intentional



03

The 4 Core Terms

Store · Reducer · Action · Dispatch

4 Terms to Master Redux

Understand these and you understand Redux

01 Store

The central warehouse.
All application data lives here in one JS object.

02 Reducer

A function that decides how state changes.
Receives old state → returns new state.

03 Action

An object you pass to dispatch.
Tells Redux what change to make (type + payload).

04 Dispatch

The trigger. You call this to start
a state change. Redux handles the rest.

Store

A single JavaScript object that holds ALL your application's data

Why a single object?

- One place for all data — easy to track
- Access any data by its key name
- One source of truth — no inconsistency
- Never modified directly — must go through reducer

E-Commerce Store Example:

```
{  
  products: [ ],  
  // list of products  
  
  orders: [ ],  
  // user orders  
  
  cart: [ ],  
  // items in cart  
  
  user: { },  
  // logged-in user  
}
```

Reducer

A function that takes old state + action → returns new state

```
reducer ( previousState , action ) → returns newState
```

Rules:

- Redux passes previousState automatically — you never supply it
- You pass action via dispatch()
- Whatever you RETURN becomes the new state
- NEVER modify the original state — return a new object
- Always include a default: return state case

```
function reducer(state, action){  
    returns newState  
}
```

Action

The input you pass to dispatch — tells Redux what change to make

Always a plain object with type:

```
{
  type:    'DEPOSIT', // required
  payload: 500        // data
}
```

type

REQUIRED
What to do:
'DEPOSIT'
'UPDATE_MOBILE'

payload

OPTIONAL
The data needed:
500 (amount)
'9123456789'

Convention: UPPER_SNAKE_CASE

'DEPOSIT'

Add money to balance

'UPDATE_MOBILE'

Change mobile number

'REMOVE_ITEM'

Remove from cart

'SET_USER'

Set logged-in user

Dispatch

The trigger — call this to start a state change

You call `dispatch()` → Redux calls your `reducer()` → Store updates

What dispatch does:

- 1 Receives the action object from you
- 2 Passes it into the Redux system internally
- 3 Redux calls `reducer(currentState, action)`
- 4 Reducer returns new state
- 5 Store is updated automatically

```
store.dispatch({  
  type: 'DEPOSIT',  
  payload: 500  
});
```

Reading state — `getState()`:

```
store.getState()  
// returns current state object
```

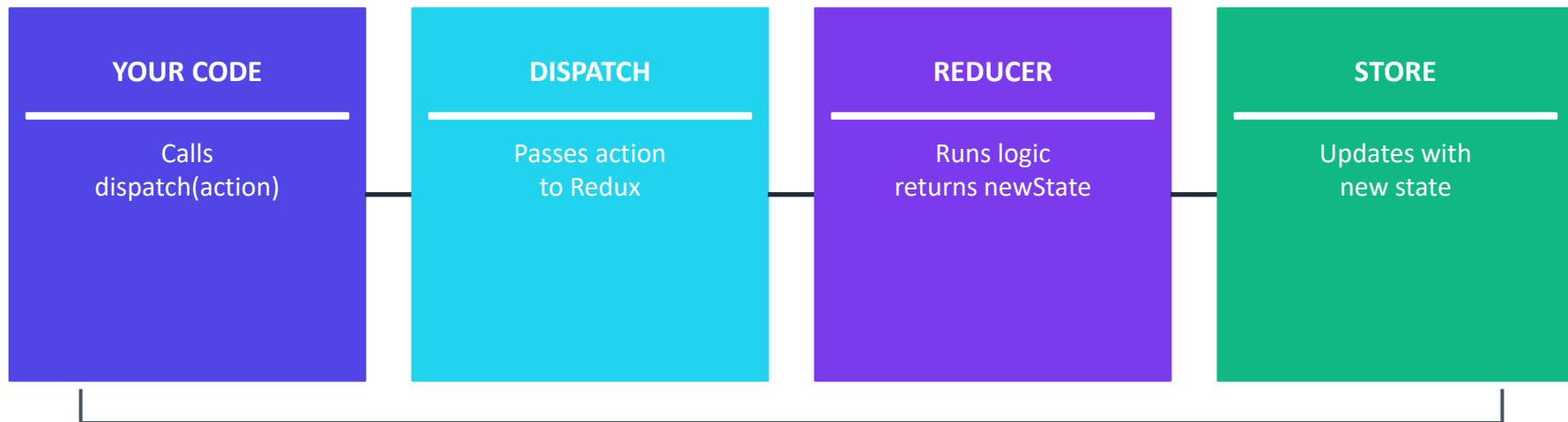

04

The Complete Redux Flow

How all 4 terms connect together

Redux Data Flow

One direction — always the same cycle



App reads updated state via `store.getState()` — components reflect new data

`dispatch(action)` → `reducer(state, action)` → `return newState` → `store updates`

05

Practical — Build It From Scratch

Plain JavaScript · User Account App

Step 1 & 2 — Install + Initial State

npm install redux · Define your starting data

Install:

```
npm install redux
```

Our example — a user account:

```
const initialState = {  
  fullName: 'John Doe',  
  mobileNumber: '9876543210',  
  balance: 0  
};
```

What we'll build:

- **Install Redux** npm install redux
- **Define initial state** User object with name, mobile, balance
- **Write the reducer** Handle DEPOSIT + UPDATE_MOBILE
- **Create the store** createStore(reducer)
- **Dispatch actions** Change balance and mobile number
- **Verify with getState()** Confirm state updated correctly

Step 3 — Write the Reducer

The function that handles state changes

```
function reducer(state = initialState, action){

  switch (action.type) {

    case 'DEPOSIT':
      return {
        ...state,
        balance: state.balance + action.payload
      };

    case 'UPDATE_MOBILE':
      return {
        ...state,
        mobileNumber: action.payload
      };

    default:
      return state;
  }
}
```

state = initialState

Default value — used the very first time

...state (spread)

Copy ALL existing properties into new object

action.payload

The data passed from dispatch()

default: return state

ALWAYS include — never return undefined

Steps 4–7 — Create Store + Dispatch

Connect everything and trigger state changes

```
const { createStore } = require('redux');

// Step 4: Create the Store
const store = createStore(reducer);

// Step 5: Check initial state
console.log(store.getState());
// { fullName: 'John Doe',
//   mobileNumber: '9876543210', balance: 0 }

// Step 6: Deposit money
store.dispatch({ type: 'DEPOSIT', payload: 500 });
console.log(store.getState().balance); // 500

// Step 7: Update mobile
store.dispatch({ type: 'UPDATE_MOBILE',
  payload: '9123456789' });

// Step 8: Deposit again
store.dispatch({ type: 'DEPOSIT', payload: 10 });
console.log(store.getState().balance); // 510
```

State after each dispatch:

Initial

balance: 0, mobile: 9876...

After DEPOSIT 500

balance: 500, mobile: 9876...

After UPDATE_MOB

balance: 500, mobile: 9123...

After DEPOSIT 10

balance: 510, mobile: 9123...

Common Mistakes to Avoid

Learn from these before you make them

Mutating state directly

✗ `state.balance = 500`

✓ `return { ...state, balance: 500 }`

Calling reducer yourself

✗ `reducer(state, action)`

✓ `store.dispatch({ type: 'X' })`

No default case

✗ `switch(action.type){ case 'X':...}`

✓ Always add: `default: return state`

Wrong dispatch input

✗ `store.dispatch('DEPOSIT')`

✓ `store.dispatch({ type: 'DEPOSIT' })`

Forgetting reducer param

✗ `createStore()`

✓ `createStore(reducer)`

Quick Recap — The 4 Terms

If you remember one thing, remember this flow

Store

One JS object. Holds ALL app data. Read with `store.getState()`

Reducer

A function. Takes `(state, action)`. Returns new state.

Action

Plain object with type + payload. Describes the change.

Dispatch

`store.dispatch(action)`. The trigger. Calls reducer for you.

The Redux Cycle

```
dispatch(action) → reducer(state, action) → return newState →  
store updates
```

One Store

Single source of truth for all app data

One Way

State only changes via dispatch → reducer

Predictable

Every change is traceable and intentional

Thank you